

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1.A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

A.Design of a SOAP-Based Hospital Management Web Service

A SOAP-based web service can provide a standardized way for hospital departments—such as reception, outpatient services, laboratories, and medical records—to exchange patient information securely and reliably.

1. Overall Architecture

The basic communication model is:

Client Application → SOAP Request → Hospital Web Service → SOAP Response → Client Application

For example:

- The **Reception System** registers a new patient.
- The **Appointment System** schedules an appointment.

- The **Doctor's Application** requests a patient's medical records.
- The SOAP service processes these requests and communicates with the hospital database.

2. Main Web Service Operations

Operation	Purpose
<code>addPatient()</code>	Register a new patient
<code>getPatient()</code>	Retrieve basic patient information
<code>updatePatient()</code>	Update patient details
<code>scheduleAppointment()</code>	Create an appointment
<code>getAppointments()</code>	Retrieve a patient's appointments
<code>addMedicalRecord()</code>	Add a diagnosis, prescription, or treatment record
<code>getMedicalRecords()</code>	Retrieve a patient's medical history
<code>AHospitalService</code>	could expose operations such as:Example service interface:
<code>HospitalService</code>	

|— addPatient(patientDetails)
|— getPatient(patientId)
|— updatePatient(patientDetails)
|— scheduleAppointment(appointmentDetails)
|— getAppointments(patientId)
|— addMedicalRecord(recordDetails)
|— getMedicalRecords(patientId)

The service contract can be described using **WSDL (Web Services Description Language)**. WSDL specifies the available operations, message formats, data types, and service endpoint, allowing different hospital systems to interoperate.

3. Structure of a SOAP Message

A SOAP message is an XML document with three important components:

SOAP Envelope

The **Envelope** is the root element. It identifies the XML document as a SOAP message and contains the Header and Body.

```
<soap:Envelope>  
  
  <soap:Header>  
  
    ...  
  
  </soap:Header>  
  
  <soap:Body>  
  
    ...  
  
  </soap:Body>  
  
</soap:Envelope>
```

SOAP Header

The **Header** contains optional metadata related to the request, such as:

- Authentication credentials or security tokens
- Digital signatures
- User or department identification
- Transaction information
- Message timestamps

Example:

```
<soap:Header>

  <Security>

    <UsernameToken>

      <Username>doctor01</Username>

      <Password>*****</Password>

    </UsernameToken>

  </Security>

</soap:Header>
```

In a real system, standards such as **WS-Security** can be used instead of sending passwords as plain text.

SOAP Body

The **Body** contains the actual request or response data.

For example, a request to register a patient could contain:

```
<soap:Body>

  <addPatient>

    <patient>

      <name>Ravi Kumar</name>

      <dateOfBirth>1990-05-12</dateOfBirth>

      <gender>Male</gender>

      <phone>9876543210</phone>
```

</patient>

</addPatient>

</soap:Body>

4. Example: Adding a Patient

Client SOAP Request

The client sends the following SOAP request to the hospital server:

<soap:Envelope

xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

<soap:Header>

<Authentication>

<UserID>reception01</UserID>

<Timestamp>2026-08-25T10:30:00Z</Timestamp>

</Authentication>

</soap:Header>

<soap:Body>

<addPatient>

<patient>

<name>Ravi Kumar</name>

<dateOfBirth>1990-05-12</dateOfBirth>

<gender>Male</gender>

<phone>9876543210</phone>

</patient>

</addPatient>

</soap:Body>

</soap:Envelope>

Server Processing

The SOAP server:

1. Receives the request.
2. Validates the SOAP message.
3. Authenticates and authorizes the user.
4. Validates patient information.
5. Stores the patient details in the hospital database.
6. Generates a unique patient ID.
7. Sends a SOAP response.

Server SOAP Response

<soap:Envelope

xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

<soap:Body>

<addPatientResponse>

<status>Success</status>

<patientId>P100245</patientId>

<message>Patient registered successfully</message>

</addPatientResponse>

</soap:Body>

</soap:Envelope>

5. Example: Retrieving Medical Records

A doctor may request a patient's medical records using the patient ID.

Request

<soap:Envelope

```
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
```

```
<soap:Header>
```

```
  <Authentication>
```

```
    <UserID>doctor01</UserID>
```

```
    <Role>Doctor</Role>
```

```
  </Authentication>
```

```
</soap:Header>
```

```
<soap:Body>
```

```
  <getMedicalRecords>
```

```
    <patientId>P100245</patientId>
```

```
  </getMedicalRecords>
```

```
</soap:Body>
```

```
</soap:Envelope>
```

Response

```
<soap:Envelope
```

```
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
```

```
  <soap:Body>
```

```
    <getMedicalRecordsResponse>
```

```
      <patientId>P100245</patientId>
```

```
      <records>
```

```
        <record>
```

```
          <date>2026-07-10</date>
```

```
          <diagnosis>Hypertension</diagnosis>
```

```

        <treatment>Medication prescribed</treatment>

    </record>

    <record>

        <date>2026-08-15</date>

        <diagnosis>Routine Checkup</diagnosis>

        <treatment>Follow-up advised</treatment>

    </record>

</records>

</getMedicalRecordsResponse>

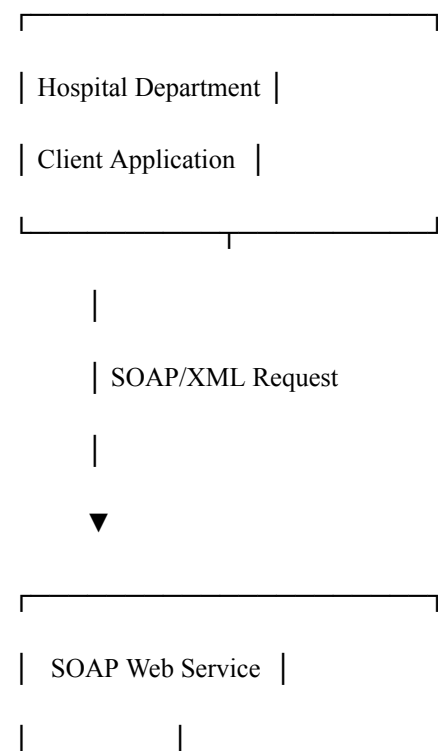
</soap:Body>

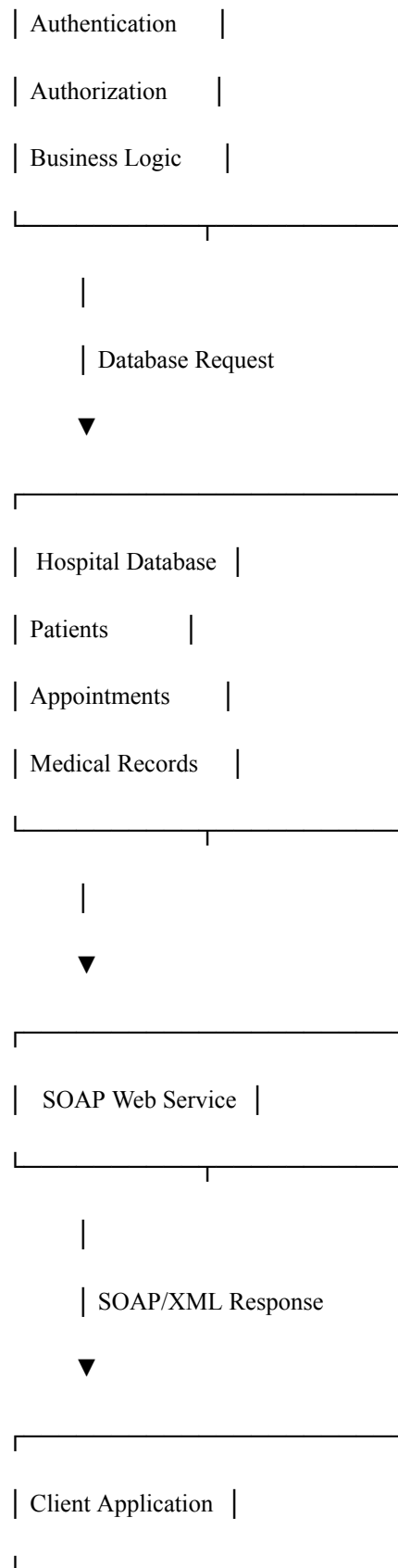
</soap:Envelope>

```

If an error occurs, the service can return a **SOAP Fault**, for example when a patient does not exist or a user lacks authorization.

6. Data Exchange Flow





7. Why SOAP Is Suitable for a Hospital System

Reliability

Hospital operations often involve important transactions. SOAP supports enterprise standards that can help ensure reliable message delivery and transaction handling. This is useful for operations such as registering patients and updating medical records.

Security

Medical information is highly sensitive. SOAP supports standards such as:

- **WS-Security**
- XML Encryption
- XML Digital Signatures
- Security tokens and authentication

These mechanisms can protect confidentiality, verify message integrity, and authenticate users. SOAP can also be used with **HTTPS/TLS** for secure transport.

Interoperability

Different departments may use different technologies—for example, Java, .NET, or legacy enterprise applications. SOAP uses standardized XML messages and WSDL service contracts, making communication between heterogeneous systems easier.

Strict Structure

SOAP provides a clearly defined message structure through the Envelope, Header, and Body. This makes messages easier to validate and process consistently across systems.

Error Handling

SOAP provides a standard **Fault** mechanism for reporting errors such as:

- Invalid patient ID
- Unauthorized access
- Invalid appointment date
- Server or database errors

Conclusion

A SOAP-based Hospital Management Web Service can expose operations such as `addPatient()`, `scheduleAppointment()`, and `getMedicalRecords()`. Clients communicate with the server using structured XML SOAP messages, where the **Envelope** defines the message, the **Header** carries metadata and security information, and the **Body** contains the actual request or response data.

SOAP is particularly appropriate for this scenario because hospital systems require **secure, reliable, standardized, and interoperable communication** between multiple departments and potentially different software platforms.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

A. Design of a WSDL Structure for an Enterprise Web Service

WSDL (Web Services Description Language) is an XML-based language used to describe a web service and explain how external applications can interact with it. It acts as a **contract** between the service provider and service consumers.

For example, an enterprise service called **DataService** may provide the following operations:

- `getData()` — retrieves data using an ID.
- `updateData()` — updates existing data.

A typical WSDL defines the **data types, messages, operations, communication protocol, and service endpoint**.

1. Overall WSDL Structure

The main components of WSDL are:

WSDL Definitions

|

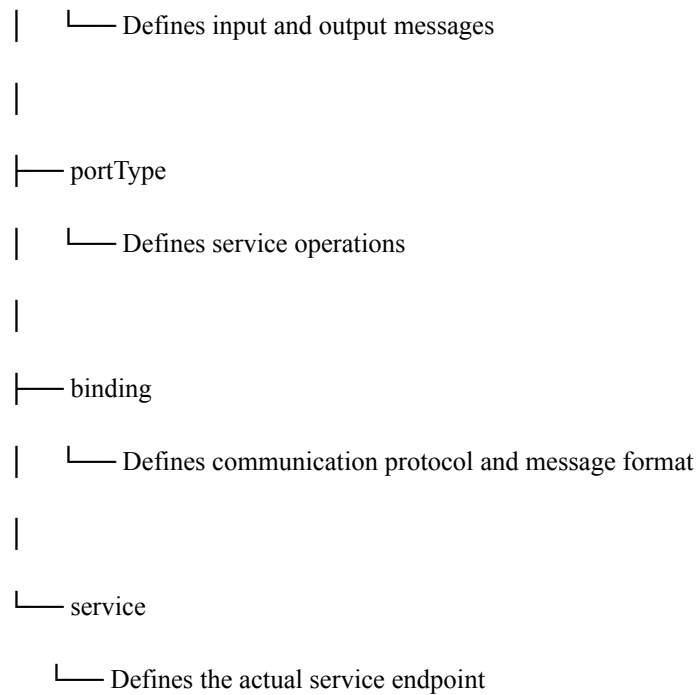
|— types

|

└— Defines XML data types

|

|— message



These components work together to provide a complete description of the web service.

2. Example WSDL Design

The following is a simplified WSDL 1.1 structure for the `DataService`:

```
<definitions
  name="DataService"
  targetNamespace="http://example.com/dataservice"
  xmlns:tns="http://example.com/dataservice"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.xmlsoap.org/wsdl/">
  <!-- 1. Types -->
  <types>
    <xsd:schema
      targetNamespace="http://example.com/dataservice">
```

```
<xsd:element name="getDataRequest">

  <xsd:complexType>

    <xsd:sequence>

      <xsd:element name="dataId"

        type="xsd:string"/>

    </xsd:sequence>

  </xsd:complexType>

</xsd:element>

<xsd:element name="getDataResponse">

  <xsd:complexType>

    <xsd:sequence>

      <xsd:element name="data"

        type="xsd:string"/>

    </xsd:sequence>

  </xsd:complexType>

</xsd:element>

<xsd:element name="updateDataRequest">

  <xsd:complexType>

    <xsd:sequence>

      <xsd:element name="dataId"

        type="xsd:string"/>

      <xsd:element name="newValue"

        type="xsd:string"/>

    </xsd:sequence>

  </xsd:complexType>

</xsd:element>
```

```

        </xsd:sequence>

    </xsd:complexType>

</xsd:element>

<xsd:element name="updateDataResponse">

    <xsd:complexType>

        <xsd:sequence>

            <xsd:element name="status"

                type="xsd:string"/>

        </xsd:sequence>

    </xsd:complexType>

</xsd:element>

</xsd:schema>

</types>

<!-- 2. Messages -->

<message name="GetDataRequest">

    <part name="parameters"

        element="tns:getDataRequest"/>

</message>

<message name="GetDataResponse">

    <part name="parameters"

        element="tns:getDataResponse"/>

</message>

<message name="UpdateDataRequest">

```

```

    <part name="parameters"
        element="tns:updateDataRequest"/>
</message>

<message name="UpdateDataResponse">

    <part name="parameters"
        element="tns:updateDataResponse"/>

</message>

<!-- 3. Port Type -->

<portType name="DataServicePortType">

    <operation name="getData">

        <input message="tns:GetDataRequest"/>

        <output message="tns:GetDataResponse"/>

    </operation>

    <operation name="updateData">

        <input message="tns:UpdateDataRequest"/>

        <output message="tns:UpdateDataResponse"/>

    </operation>

</portType>

<!-- 4. Binding -->

<binding name="DataServiceSOAPBinding"

    type="tns:DataServicePortType">

    <soap:binding style="document"

```

```
        transport="http://schemas.xmlsoap.org/soap/http"/>

    <operation name="getData">

        <soap:operation soapAction="getData"/>

        <input>

            <soap:body use="literal"/>

        </input>

        <output>

            <soap:body use="literal"/>

        </output>

    </operation>

    <operation name="updateData">

        <soap:operation soapAction="updateData"/>

        <input>

            <soap:body use="literal"/>

        </input>

        <output>

            <soap:body use="literal"/>

        </output>

    </operation>

</binding>

<!-- 5. Service -->

<service name="DataService">

    <port name="DataServicePort"
```

```
        binding="tns:DataServiceSOAPBinding">

        <soap:address

        location="https://example.com/services/DataService"/>

    </port>

</service>

</definitions>
```

3. Key Components of WSDL

A. types

The **types** section defines the data structures used by the web service. It usually uses **XML Schema (XSD)**.

For example:

```
<xsd:element name="updateDataRequest">

    <xsd:complexType>

        <xsd:sequence>

            <xsd:element name="dataId" type="xsd:string"/>

            <xsd:element name="newValue" type="xsd:string"/>

        </xsd:sequence>

    </xsd:complexType>

</xsd:element>
```

This defines the format of data sent when updating information.

Purpose: Ensures that both the client and server understand the structure and data types of exchanged information.

B. message

The **message** element defines the data exchanged between the client and the web service.

Example:

```
<message name="GetDataRequest">  
  <part name="parameters"  
    element="tns:getDataRequest"/>  
</message>
```

A message may represent:

- An input request
- An output response
- A fault or error message

For example:

Client

|
| GetDataRequest



Web Service

|
| GetDataResponse



Client

Purpose: Defines what information is sent and received for each operation.

C. portType

The **portType** defines the abstract interface of the web service. It specifies the operations that clients can invoke.

Example:

```
<portType name="DataServicePortType">  
  <operation name="getData">
```

```

    <input message="tns:GetDataRequest"/>

    <output message="tns:GetDataResponse"/>

</operation>

<operation name="updateData">

    <input message="tns:UpdateDataRequest"/>

    <output message="tns:UpdateDataResponse"/>

</operation>

</portType>

```

This is similar to an **interface in object-oriented programming**.

It tells the client:

- Which operations are available
- What request each operation accepts
- What response each operation returns

Purpose: Defines **what the service can do**, independent of how communication is implemented.

D. binding

The **binding** section specifies how the operations and messages are transmitted over the network.

Example:

```

<binding name="DataServiceSOAPBinding"

    type="tns:DataServicePortType">

    <soap:binding style="document"

        transport="http://schemas.xmlsoap.org/soap/http"/>

</binding>

```

The binding defines details such as:

- Communication protocol, such as HTTP
- Message format, such as SOAP
- SOAP style, such as document or RPC
- Encoding rules

For example:

Abstract Interface

|



portType

|

| Binding specifies HOW



SOAP + HTTP

Purpose: Connects the abstract service operations with a concrete communication protocol.

E. service

The **service** section specifies where the web service is available.

Example:

```
<service name="DataService">
```

```
  <port name="DataServicePort"
```

```
    binding="tns:DataServiceSOAPBinding">
```

```
    <soap:address
```

```
      location="https://example.com/services/DataService"/>
```

```
  </port>
```

```
</service>
```

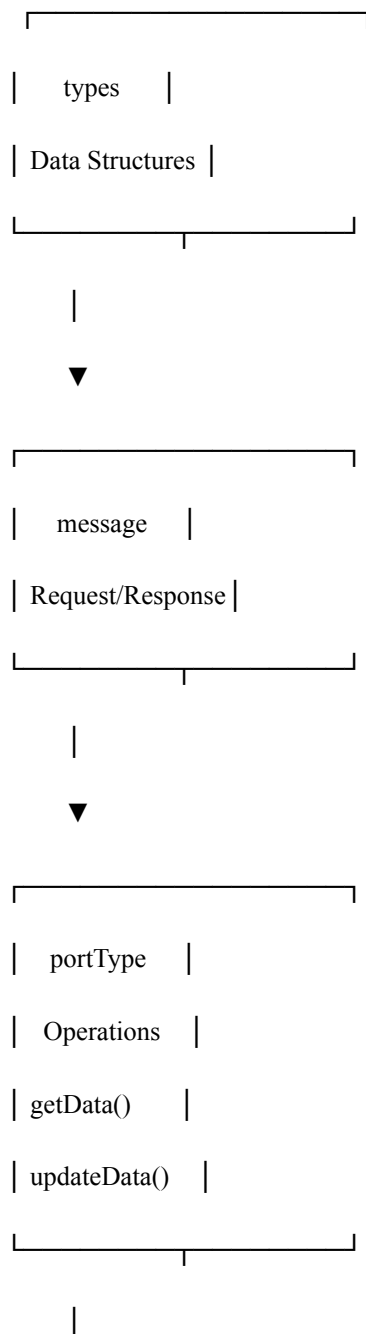
It contains:

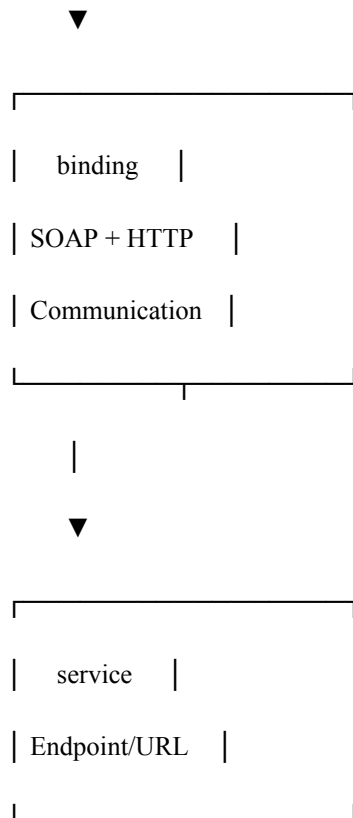
- A **service** name
- One or more **ports**
- A binding associated with each port
- The physical endpoint address

Purpose: Tells clients **where to access the service**.

4. How the Components Work Together

The relationship between WSDL components can be illustrated as follows:





In simple terms:

Types define the data → Messages package the data → PortType defines operations → Binding defines communication → Service provides the endpoint.

5. Example of Service Interaction

Suppose an external client wants to retrieve information.

Step 1: Read the WSDL

The client examines the WSDL and discovers:

- Operation: `getData`
- Input: `GetDataRequest`
- Output: `GetDataResponse`
- Protocol: SOAP over HTTP
- Endpoint: Defined in the `service` section

Step 2: Send a SOAP Request

`<soap:Envelope>`

```
<soap:Body>

  <getDataRequest>

    <dataId>D1001</dataId>

  </getDataRequest>

</soap:Body>

</soap:Envelope>
```

Step 3: Service Processes the Request

The server receives the request, retrieves the required data, and prepares a response.

Step 4: Receive the Response

```
<soap:Envelope>

  <soap:Body>

    <getDataResponse>

      <data>Enterprise Data Value</data>

    </getDataResponse>

  </soap:Body>

</soap:Envelope>
```

The WSDL ensures that both the client and server already agree on the required message format and operations.

6. Importance of WSDL in Service Integration

WSDL is important for enterprise service integration for the following reasons:

1. Standardized Service Contract

WSDL provides a formal contract between the service provider and external clients. Both sides know exactly how to communicate.

2. Interoperability

Systems developed using different technologies, such as Java, .NET, or other platforms, can interact because the WSDL describes the service using standardized XML-based definitions.

3. Automatic Client Generation

Development tools can use a WSDL document to automatically generate client-side code or service stubs, reducing development effort and errors.

4. Clear Separation of Interface and Implementation

The **portType** describes **what** the service does, while the **binding** describes **how** it communicates. This separation makes services easier to modify and integrate.

5. Reduced Integration Errors

Clearly defined message structures and data types ensure that clients send correctly formatted requests and can correctly interpret responses.

Conclusion

WSDL provides a complete and structured description of a web service. Its main components are:

- **types** — defines data types and structures.
- **message** — defines input and output data.
- **portType** — defines the available operations.
- **binding** — specifies how communication takes place.
- **service** — specifies the service endpoint.

Together, these components form a **service contract** that allows external systems to discover, understand, and communicate with the web service. This makes WSDL particularly valuable in enterprise environments where reliable integration and interoperability between different applications and technologies are required.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

A. Analysis and Debugging of SOAP Communication Issues

A SOAP-based system follows a structured request-response process. If requests are not processed correctly or responses are delayed, the problem may occur in the **SOAP message, namespace configuration, endpoint, network connection, server processing, or error handling**.

1. SOAP Communication Process

The normal SOAP communication flow is:

Client Application

|

| 1. Create SOAP Request



SOAP Envelope

|— Header

└— Body

|

| 2. Send using HTTP/HTTPS



Web Service Endpoint

|

| 3. Validate and Process Request



Application / Database

|

| 4. Generate SOAP Response



SOAP Response

|



Client Application

The debugging process should examine each stage to identify where communication is failing.

2. Possible SOAP Communication Problems

A. Incorrect SOAP Message Formatting

SOAP messages must follow the required XML structure. Common problems include:

- Missing **Envelope** element
- Missing or incorrectly placed **Header** or **Body**
- Invalid XML syntax
- Unclosed XML tags
- Incorrect element names
- Invalid data types
- Incorrect request structure

Incorrect Example

```
<soap:Envelope>  
  
  <soap:Body>  
  
    <getCustomer>  
  
      <customerId>1001</customerId>  
  
    </getCustomer  
  
  </soap:Body>  
  
</soap:Envelope>
```

The closing `getCustomer` tag is invalid.

Corrected Example

```
<soap:Envelope
```

```
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

<soap:Body>

  <getCustomer>

    <customerId>1001</customerId>

  </getCustomer>

</soap:Body>

</soap:Envelope>
```

The XML must be **well-formed** before the SOAP service can process it.

B. Incorrect or Missing Namespaces

Namespaces are essential because they identify the SOAP version and the web service operations.

For example:

```
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
```

A common issue occurs when the SOAP version expected by the server differs from the version sent by the client.

SOAP 1.1 Namespace

```
http://schemas.xmlsoap.org/soap/envelope/
```

SOAP 1.2 Namespace

```
http://www.w3.org/2003/05/soap-envelope
```

If a SOAP 1.1 service receives a SOAP 1.2 message, it may reject the request.

The application-specific namespace must also match the WSDL definition.

Example

```
<soap:Body>

  <ns:getCustomer
```

```
xmlns:ns="http://example.com/customer">  
  
<customerId>1001</customerId>  
  
</ns:getCustomer>  
  
</soap:Body>
```

If the client uses the wrong namespace, the server may be unable to recognize `getCustomer`.

C. Incorrect Service Endpoint

The client must send the SOAP request to the correct endpoint.

For example:

`https://api.example.com/services/CustomerService`

Possible endpoint problems include:

- Incorrect URL
- Wrong port number
- Using HTTP instead of HTTPS
- Service deployed at a different path
- Server unavailable
- DNS failure
- Firewall blocking the connection

The endpoint defined in the client's configuration should be compared with the endpoint specified by the current WSDL.

D. Incorrect HTTP Headers

SOAP communication often requires specific HTTP headers.

For SOAP 1.1, a typical request may include:

Content-Type: text/xml; charset=utf-8

SOAPAction: "getCustomer"

For SOAP 1.2, the content type is typically different.

A missing or incorrect `SOAPAction` can cause some SOAP servers to reject the request or invoke the wrong operation.

E. WSDL and Client-Service Contract Mismatch

The client may be using an outdated version of the WSDL.

For example:

Client expects: `getCustomer(customerId)`

Actual service expects: `getCustomer(customerNumber)`

Other mismatches may involve:

- Changed operation names
- Modified XML schemas
- Different namespaces
- Required parameters missing
- Changed endpoint addresses

This can result in invalid or incorrect responses.

F. Delayed Responses and Timeout Problems

If responses are delayed, the problem may not be in the SOAP XML itself.

Possible causes include:

- Slow database queries
- High server load
- Network latency
- Long-running operations
- Connection timeout settings
- Insufficient server resources
- External services responding slowly

The system should measure how long each stage takes:

Request Creation → 20 ms

Network Transfer → 50 ms

SOAP Validation → 10 ms

Business Processing → 5000 ms ← Possible problem

Database Query → 4500 ms ← Investigate

Response Transfer → 40 ms

This helps identify the actual source of the delay.

3. Recommended Debugging Approach

A systematic debugging process should be followed.

Step 1: Capture the Complete SOAP Request and Response

Log the raw SOAP messages sent and received.

Check:

- Request XML
- Response XML
- HTTP headers
- Endpoint URL
- HTTP status codes
- SOAP faults

Example log:

Request Time: 10:30:15

Endpoint: <https://server.example.com/CustomerService>

Operation: getCustomer

HTTP Status: 500

SOAP Fault: Invalid namespace

Sensitive information such as passwords and security tokens should be masked in production logs.

Step 2: Validate the XML Structure

Validate whether the SOAP request is well-formed XML.

Check that:

- Every opening tag has a closing tag.
- Elements are correctly nested.
- Attribute values are properly quoted.

- Special XML characters are escaped.
- Only valid XML characters are used.

For example:

```
<customerName>John & Smith</customerName>
```

is invalid XML because **&** must be escaped.

Correct version:

```
<customerName>John &amp; Smith</customerName>
```

Step 3: Validate Against the WSDL and XSD

The SOAP request should be compared with the service contract.

Verify:

- Operation name
- Namespace URI
- Required elements
- Element order, where required
- Data types
- Request and response structure

For example, if the schema requires:

customerId → integer

then sending:

```
<customerId>ABC123</customerId>
```

may cause validation failure.

Using XML Schema validation helps detect these problems before the request reaches business logic.

Step 4: Verify Namespaces and SOAP Version

Check the namespace declarations carefully.

A corrected message may look like:

```
<soapenv:Envelope
```

```
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
```

```
xmlns:cust="http://example.com/customer">
```

```
<soapenv:Header/>
```

```
<soapenv:Body>
```

```
  <cust:getCustomer>
```

```
    <cust:customerId>1001</cust:customerId>
```

```
  </cust:getCustomer>
```

```
</soapenv:Body>
```

```
</soapenv:Envelope>
```

The namespace URI must match the URI specified by the service contract.

Step 5: Test the Endpoint Independently

Use a SOAP testing client to send a known valid request directly to the service.

Verify:

- ✓ Server reachable
- ✓ Correct URL
- ✓ Correct port
- ✓ HTTPS certificate valid
- ✓ Service running
- ✓ Operation available

If a valid request succeeds in a testing tool but fails in the application, the problem is likely in the client configuration or message generation.

Step 6: Check Server-Side Logs

Server logs should be examined to determine whether the request:

1. Never reached the server.
2. Reached the endpoint but failed SOAP parsing.
3. Passed parsing but failed schema validation.
4. Reached the application logic but caused an exception.
5. Failed while accessing the database or another service.

This distinction is important because each failure requires a different solution.

4. SOAP Fault Handling

SOAP provides a standard mechanism called a **SOAP Fault** for reporting errors.

Example:

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

  <soap:Body>

    <soap:Fault>

      <faultcode>soap:Client</faultcode>

      <faultstring>

        Invalid customerId format

      </faultstring>

    </soap:Fault>

  </soap:Body>

</soap:Envelope>
```

Common fault categories include:

Fault Type	Meaning
Client	The client sent an invalid request

Server A server-side processing error occurred

Authentication fault User credentials or security token is invalid

Validation fault Data does not match the expected schema

Endpoint fault Requested service or operation is unavailable

The client should parse SOAP Faults instead of treating every error as an unknown failure.

5. Corrected Communication Approach

After debugging, the communication process should follow a controlled sequence:

1. Client creates request

|



2. XML validation

|



3. XSD/WSDL contract validation

|



4. Add correct namespaces and security headers

|



5. Send to verified HTTPS endpoint

|



6. Server validates SOAP message



7. Server executes business operation



8. Server returns valid SOAP response or SOAP Fault



9. Client validates and processes response

A corrected SOAP request could be:

```
<soapenv:Envelope
```

```
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
```

```
  xmlns:cust="http://example.com/customer">
```

```
    <soapenv:Header>
```

```
      <cust:RequestID>REQ-1001</cust:RequestID>
```

```
    </soapenv:Header>
```

```
    <soapenv:Body>
```

```
      <cust:getCustomer>
```

```
        <cust:customerId>1001</cust:customerId>
```

```
      </cust:getCustomer>
```

```
    </soapenv:Body>
```

</soapenv:Envelope>

6. How the Corrected System Ensures Reliable Communication

The corrected system improves reliability through several mechanisms:

Message Validation

XML and XSD validation prevent malformed or invalid messages from being processed.

Contract Consistency

Using the correct WSDL ensures that the client and server agree on operations, parameters, namespaces, and data formats.

Endpoint Verification

Correct endpoint configuration ensures that requests reach the intended service.

Standardized Error Handling

SOAP Faults provide structured and understandable error information.

Logging and Monitoring

Request IDs, timestamps, response times, and fault logs make communication problems easier to trace and diagnose.

Timeout and Retry Management

Appropriate timeout values and carefully controlled retries can handle temporary communication failures. For update operations, retries should use an idempotency or request identifier mechanism where possible to avoid processing the same request twice.

Security

HTTPS and SOAP security mechanisms protect messages from unauthorized access or modification.

Conclusion

SOAP communication failures can result from **malformed XML, incorrect namespaces, SOAP-version mismatches, invalid HTTP headers, outdated WSDL contracts, wrong**

endpoints, server errors, or performance bottlenecks. A systematic debugging approach should capture raw requests and responses, validate XML and schemas, verify namespaces and endpoints, inspect HTTP headers and server logs, and handle SOAP Faults correctly.

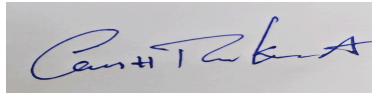
By validating messages against the service contract, using correct endpoint and namespace configurations, implementing structured error handling, and monitoring the complete request-response process, the corrected SOAP system provides **more reliable, accurate, secure, and maintainable communication** between clients and web services.

Code of Conduct:

*I **Abburi Ganesh Ram Kamal 192311183** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

